

EE-2003

Computer Organization and Assembly Language

Week 12 – Lecture 23-24

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Chapter 6: Display Memory (Introduction)
- 3 ASCII Codes – Introduction
- 4 Display Memory Formation
- 5 Mapping the Display Memory
- 6 Understanding the Attribute Byte
- 7 Example: Clearing the Screen

Quick Review: What was in the Last Week?

- Local Variables on the stack
- Quick Review of Chapter 5

Chapter 6: Display Memory (Introduction)

- In earlier programs, we used the **debugger** to monitor processor operations instruction by instruction.
- The debugger gives us a close view of the processor — helping detect bugs and understand execution flow.
- So far, all programs have executed within the debugger, but now we move toward **interacting with real hardware**.
- In this chapter, we will begin using the **display screen in character mode**, the same way DOS handles screen output.

We'll now explore how data is shown on your screen — the visible output of your code!

Display Memory and the DOS Environment

- The display screen in DOSBox acts as your **console** — a virtual screen where your 16-bit assembly programs run.
- DOSBox serves as an **emulator**, mimicking the real IBM PC hardware environment.
- Every time you run a program, DOS loads it into memory, executes it, and then steps aside — allowing your code to directly control the screen memory.
- This gives you the freedom to explore how text and data are displayed at the hardware level.

DOS provides just enough support to load your program — the rest is under your control!

Why Study Display Memory?

- Understanding display memory bridges the gap between **software and hardware**.
- When working with display memory, you are directly writing to the screen buffer — no OS or API in between.
- These concepts are essential for:
 - **High-performance computing** (where speed and control are crucial)
 - **Game development** (direct pixel and character manipulation)
 - **Embedded and system-level programming**
- In modern systems, the OS manages these tasks — but in DOS, you interact with the screen **directly**.

Mastering display memory helps you understand how your code truly “talks” to hardware.

Chapter Overview

- Explore the concept of **video memory** and its address mapping.
- Learn how to write characters directly to the screen using assembly instructions.
- Understand the difference between:
 - **Character mode** (text display)
 - **Graphics mode** (pixel display)
- Study how DOS and BIOS manage screen operations.

Goal: By the end of this chapter, you'll know how to display data directly on screen without using DOS interrupts.

6.1 ASCII Codes – Introduction

- The computer does not understand letters or symbols — it only understands **numbers (binary values)**.
- Every key you press on the keyboard sends a specific number to the computer.
- For example, when you press '**A**', the keyboard sends a numeric code.
- The **VGA (Video Graphics Adapter)** converts that number into the visible shape of the letter 'A' on the screen.
- Each glowing pixel you see is represented by a binary bit:
 - 1 → Pixel ON
 - 0 → Pixel OFF

Everything you see or type — even letters — are just numbers inside the computer!

The Need for ASCII

- To make sure all computers interpret characters the same way, a standard was created — the **ASCII Code**.
- **ASCII** stands for *American Standard Code for Information Interchange*.
- It allows the same letter (like 'A') to look identical on every computer and device.
- This made it possible for computers to communicate using a common standard.
- Although there are other encoding systems like **EBCDIC** and **Gray Code**, ASCII became the most widely used — especially on the Internet and DOS-based systems.

ASCII made communication between computers simple and consistent.

Standard and Extended ASCII

- **Standard ASCII** has **128 characters** (numbers 0–127).
- It includes:
 - Alphabets (A–Z, a–z)
 - Numbers (0–9)
 - Punctuation marks and control characters
- When the **IBM PC** was introduced, it extended ASCII by adding 128 more characters.
- This created the **Extended ASCII** set (0–255) — fitting perfectly in one 8-bit byte.
- The new symbols were used for:
 - Line drawing and window borders
 - Non-English letters
 - Special graphical symbols

Extended ASCII allowed early PCs to show boxes, menus, and borders — even before graphics mode existed!

Using ASCII Codes in Programming

- ASCII codes are stored and processed as **numbers**, making text handling simple in assembly.
- Important number ranges in ASCII:
 - **Digits:** 0x30 – 0x39 (0–9)
 - **Uppercase letters:** 0x41 – 0x5A (A–Z)
 - **Lowercase letters:** 0x61 – 0x7A (a–z)
- These continuous ranges make conversions easy:
 - Convert uppercase → lowercase: add 0x20
 - Convert lowercase → uppercase: sub 0x20
 - Convert number → ASCII digit: add 0x30

Knowing ASCII helps you control how data is displayed and stored in assembly programs.

6.2 Display Memory Formation – Introduction

- This section explains how the screen displays characters using **ASCII codes**.
- Sending a value like 0x41 to the VGA card makes it display the letter 'A'.
- Each pixel on the screen lights up according to binary data.
- The **VGA (Video Graphics Adapter)** converts numbers into shapes on screen.
- The display screen is just part of memory — what you write there becomes visible.

"The screen is memory made visible!"

Video Memory Concept

- The display memory stores ASCII codes for what is shown on the screen.
- Each character cell = 2 bytes: one for ASCII and one for color or style.
- Text screen size: 80 columns $\times$ 25 rows = 2000 characters.
- The first 80 words correspond to the first row, the next 80 to the next row.

Screen Layout (Simplified):

Row1: [C1] [C2] ... [C80]

Row2: [C1] [C2] ... [C80]

...

Row25: [C1] [C2] ... [C80]

Each [C] = ASCII + Attribute (2 bytes)

Mapping the Display Memory

- Video memory (text mode) starts at address 0xB8000.
- Each word has:
 - Lower byte → ASCII code
 - Upper byte → Attribute (color, blink, intensity)
- ES is commonly used to point to video memory.

Example Setup:

```
MOV AX, 0xB800  
MOV ES, AX
```

Now ES:DI directly controls what appears on the screen.

Writing Characters to the Screen

- Each word = [Attribute Byte][ASCII Code]

Example:

```
MOV WORD [ES:0], 0x0741 → 'A' in white on black  
MOV WORD [ES:160], 0x1230 → '0' in green on black
```

- 41 = ASCII code for 'A'
- 07 = White text on black background
- 160 = 80×2 = Start of second line

Understanding the Attribute Byte

Attribute Byte Structure:

Bit	Description
7	Blinking (on/off)
6–4	Background color (RGB)
3	Intensity (Foreground)
2–0	Foreground color (RGB)

Example: 07 → White text on black background

Example: Clearing the Screen

Program Outline:

```
ORG 0x0100
MOV AX, 0xB800
MOV ES, AX
MOV DI, 0x0000
NEXTPOS: MOV WORD [ES:DI], 0x0741
ADD DI, 2
CMP DI, 4000
JNE NEXTPOS
MOV AH, 0x1
INT 0x21
MOV AX, 0x4C00
INT 0x21
```

This program fills the screen with the letter 'A'.

System Flow: From CPU to Screen

How Data Flows:

CPU → System Bus → Video Memory (B800h) → VGA → Screen

CPU writes data → Bus transfers it → VGA shows it → Screen displays it.

Thank You!

Any Questions?

Key Takeaways:

- The screen is just memory — what you write there becomes visible.
- Each screen cell = 2 bytes → (ASCII + Attribute).
- Video memory in text mode begins at 0xB8000.
- Direct hardware-level access gives complete control over the display.

“Display Memory shows how software meets hardware — every character on screen is a number in memory.”